A3 — Value-Function Approximation: task, goal, and how it was solved

What phase A3 asks, why the table has to go, and what the ablations showed. The lab notebook with the numbers is `experiments/a3_dqn/README.md`.

> **At a glance**
>
> - Remove DQN's target network and it diverges to a constant action (9.2, worse than random); remove replay and it merely slows down (80 against 250).
> - Full DQN is itself unstable on CartPole: it peaks at 469 and ends at 250.
> - Overestimation only appears once the network has learned (after ~16–20k steps). Double DQN halves it; on Pong it turns over-estimation into under-estimation, as van Hasselt predicts.

---

1. The task

`ladder.md` states four items:

1. DQN on CartPole/LunarLander; disable the replay buffer and target network independently, document the instability.
2. Add Double DQN; measure whether Q-value overestimation drops.
3. Add a dueling architecture; compare sample efficiency vs. vanilla DQN.
4. Optional: DQN on a small Atari environment.

Item 4 is not run here — it is the only piece that genuinely wants a GPU. The initial reasoning was that it adds no concept the other three cover; **items 2 and 3 then showed that reasoning was wrong**, for a reason recorded in section 7. It is the environment where the other two items' effects would actually be measurable.

2. The goal

A2 could store one number per state. That ends here: CartPole's state is four real numbers, so the table would need an entry per point in $\mathbb{R}^4$. The fix is to *approximate* Q with a parameterised function $Q_\theta(s, a)$ and learn θ by regression on TD targets:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right]$$

That single change breaks every guarantee A2 relied on, and it is worth being precise about how.

The precise failure. Let $\mathcal{F} = \{Q_\theta\}$ be the representable set and Π_μ the projection onto it in the norm weighted by the state distribution μ the data was drawn from. Fitted value iteration is not $Q \leftarrow TQ$ but

$$Q_{k+1} = \Pi_\mu \, T \, Q_k$$

Both pieces are well behaved *separately*: T is a γ-contraction in $\|\cdot\|_\infty$ (proved in A2), and Π_μ is a non-expansion in $\|\cdot\|_{2,\mu}$. But **they contract in different norms**, and a composition of a contraction in one norm with a non-expansion in another is not a contraction in either. Banach's theorem no longer applies; there need not even be a fixed point.

The classical positive result says exactly where the danger lies. For *on-policy* evaluation with linear function approximation — that is, when $\mu = d_\pi$, the stationary distribution of the policy being evaluated — Tsitsiklis and Van Roy (1997) showed $\Pi_{d_\pi} T^\pi$ *is* a contraction in $\|\cdot\|_{2,d_\pi}$ and TD converges. The proof turns on P_π being a non-expansion in precisely that weighted norm, which holds because d_π is P_π-invariant.

That is the crux: the guarantee is tied to sampling from the policy's own stationary distribution. Go off-policy and $\mu \neq d_\pi$, the invariance fails, and the composition can expand. Baird's counterexample (1995) makes it concrete — a seven-state MDP where linear TD with off-policy sampling diverges, with weights growing without bound. The result is the **deadly triad**:

ingredient	where it appears here
bootstrapping	the target contains Q itself
off-policy learning	the $\max$ evaluates a policy the agent is not following
function approximation	one gradient step changes Q at states it never visited

Any two are usually fine. All three together can diverge. **The goal of A3 is to find out which stabiliser addresses which ingredient** — which is exactly why item 1 removes them one at a time rather than reporting that DQN works.

3. Why the two stabilisers exist

Replay attacks correlation and distribution shift. SGD's convergence argument assumes each minibatch gradient is an unbiased, finite-variance estimate of the full-data gradient drawn independently; consecutive transitions from one episode violate independence badly, and the effective sample size of a correlated batch of n is far below n. Worse, without a buffer the training distribution moves with the policy, so the network continually overwrites what it learned on states it no longer visits. Sampling uniformly from a buffer of the last N transitions restores something closer to i.i.d. data over a slowly-changing distribution.

The target network attacks the moving target. In the loss above, θ^- is a periodically-frozen copy. Without it ($\theta^- = \theta$) the regression target moves the moment the parameters do, and because the approximator generalises, updating $Q(s,a)$ also shifts $Q(s',a')$ in the target — the network chases a value it is itself pushing away.

Note these two are answers to *different* ingredients: replay to the data distribution, the target network to bootstrapping. That is the prediction item 1 tests.

4. Double DQN (item 2)

The max operator is biased upward. For any noisy estimates,

$$\mathbb{E}\left[\max_a \hat{Q}(s,a)\right] \geq \max_a \mathbb{E}\left[\hat{Q}(s,a)\right]$$

by Jensen's inequality, since $\max$ is convex. Selecting *and* evaluating with the same noisy estimates means any action whose value is overestimated by noise is the one the max picks — the error is selected for, not averaged out.

How large is the bias? Suppose the $|\mathcal{A}|$ estimates are unbiased, independent and Gaussian with standard deviation σ. Then the expected maximum is the expected maximum of $|\mathcal{A}|$ standard normals, scaled:

$$\mathbb{E}\left[\max_a \hat{Q}(s,a)\right] - \max_a q(s,a) \;\approx\; \sigma\sqrt{2\ln|\mathcal{A}|}$$

Two predictions fall out of that expression, and both matter for reading the experiment. The bias grows with the **noise** σ — so it is small early when the network is merely wrong in a systematic way, and grows once errors become noise-like — and with the **number of actions**, only as $\sqrt{\ln|\mathcal{A}|}$ but still monotonically. CartPole has $|\mathcal{A}| = 2$, giving $\sqrt{2\ln 2} \approx 1.18\,\sigma$; Atari's 18 actions give $\approx 2.4\,\sigma$, twice as much. This is the quantitative version of why the original paper used Atari.

Why splitting the estimators helps. Let $\hat{Q}^A, \hat{Q}^B$ be two independent unbiased estimates. The double estimator uses A to select and B to evaluate:

$$a^\dagger = \arg\max_a \hat{Q}^A(s,a), \qquad \text{estimate} = \hat{Q}^B(s,a^\dagger)$$

Conditioned on $a^\dagger$, the estimate $\hat{Q}^B(s,a^\dagger)$ is unbiased for $q(s,a^\dagger)$ because B's noise played no part in choosing it. Hence

$$\mathbb{E}\left[\hat{Q}^B(s,a^\dagger)\right] = \mathbb{E}\left[q(s,a^\dagger)\right] \leq \max_a q(s,a)$$

The selection bias is gone; van Hasselt (2010) shows the estimator now *under* estimates instead, which is generally the safer direction. DQN's two networks are not independent — the target is a delayed copy of the online net — so the practical effect is a reduction in the bias, not its elimination.

Double DQN splits the two roles:

$$\text{DQN:}\quad r + \gamma\, Q_{\theta^-}\Big(s', \arg\max_{a'} Q_{\theta^-}(s',a')\Big)$$

$$\text{Double:}\quad r + \gamma\, Q_{\theta^-}\Big(s', \arg\max_{a'} Q_{\theta}(s',a')\Big)$$

The online network chooses, the target network scores. Two independent estimates are unlikely to have correlated noise on the same action, so the selected action is no longer systematically the over-valued one.

Measuring the bias. The probe is the difference between what the network believes at an episode's start state and what the greedy policy actually collects from it:

$$\text{overestimation} = \max_a Q_\theta(s_0, a) - \mathbb{E}\left[\sum_t \gamma^t r_t\right]$$

A design decision worth stating: deterministic CartPole supplies almost no estimate noise for the max to amplify, so the effect may be small there by construction. The comparison is therefore run twice — once as-is, and once with zero-mean Gaussian reward noise, which injects the variance the bias feeds on without changing the optimal policy. If Double DQN only helps in the noisy variant, that is a result about *when* the fix matters, not a failed experiment.

5. Dueling (item 3)

The dueling head splits Q into a state value and an advantage:

$$Q(s,a) = V(s) + \left(A(s,a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a') \right)$$

Subtracting the mean advantage is not cosmetic, and the reason is an identifiability argument. The map $(V, A) \mapsto Q = V + A$ has a one-dimensional gauge freedom at every state: for any $c(s)$,

$$\big(V(s) + c(s),\ A(s,a) - c(s)\big) \ \longmapsto\ \text{the same } Q(s,a)$$

so V and A are not determined by Q, and nothing in the TD loss — which only ever sees Q — constrains them. The two streams can drift arbitrarily far in opposite directions while fitting the data equally well.

Imposing $\sum_a A(s,a) = 0$ fixes the gauge and makes the decomposition unique. With the mean subtracted, averaging the recombination over actions gives

$$\frac{1}{|\mathcal{A}|}\sum_a Q(s,a) = V(s) + \frac{1}{|\mathcal{A}|}\sum_a \big(A(s,a) - \overline{A}(s)\big) = V(s)$$

so V is pinned to the mean action-value and A to the deviations from it — which is what makes the "value stream" interpretation true rather than aspirational.

The argument for it: in most states the choice of action barely matters, and the useful signal is "is this state good at all". A shared $V(s)$ stream is updated by *every* transition through s regardless of the action taken, whereas a plain head only updates the column of the action actually tried. So it should learn more per sample — a sample-efficiency claim, which is why the item asks for efficiency rather than final score.

6. What was done

`dqn.py` holds one agent with four switches — `replay`, `target_network`, `double`, `dueling` — so every arm in every item is the same code path with different flags, and no comparison can be contaminated by an incidental implementation difference.

Decisions worth recording:

- **Truncation is not termination.** CartPole ends at 500 steps by time limit. Bootstrapping is suppressed only on true termination; treating a time-limit truncation as terminal would teach the agent that the world stops at step 500 and cap every value estimate. Gymnasium returns these as separate flags and this is the most common place to get it wrong.
- **The no-replay ablation trains on the single latest transition**, not on a batch of one sampled from a buffer — the whole point is the loss of both decorrelation and the ability to revisit old data.
- **The no-target ablation bootstraps from the online network** rather than updating the target every step, which is the same thing expressed without a redundant copy.
- **Runs are parallelised across processes**, with `torch.set_num_threads(1)` inside each, since four single-threaded runs beat one four-threaded run on networks this small.
- Evaluation is **greedy** ($\varepsilon = 0$) and separate from the training environment, so exploration cost does not contaminate the score — the A2 lesson about online return versus policy quality.

7. Results

Tables in the phase README; the parts that change how the phase should be read:

Item 1 confirms the mapping in section 3, and asymmetrically. Removing the target network collapses the agent to 9.2 — the exact score of a constant "push right" policy, against 22.7 for a random one. It does not learn badly; its Q-function degenerates until the greedy action is the same everywhere, which is divergence rather than slow learning, and leaves it *worse than random*. Removing replay instead costs a lot without being fatal (80.0 against 250.1).

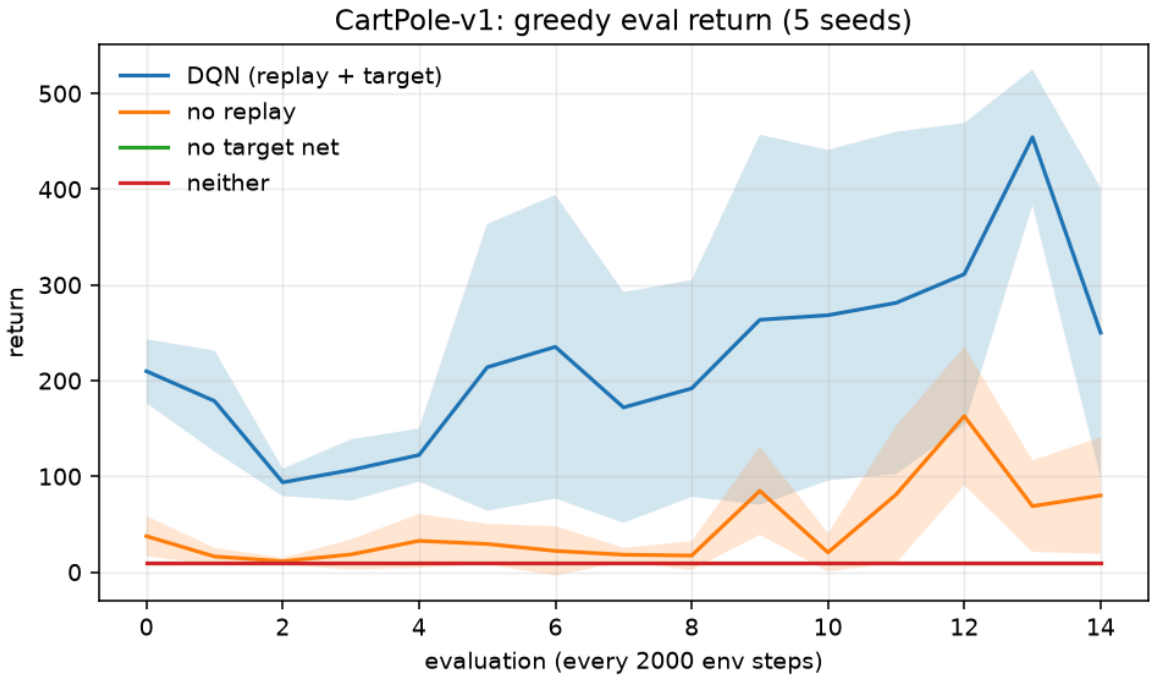


Figure — CartPole greedy evaluation return, 5 seeds: full DQN against DQN without replay, without a target network, and without either.

So the two stabilisers are not interchangeable, and the asymmetry has a reason. Replay attacks the *data* — correlation and distribution shift — and a learner with correlated data is inefficient. The target network attacks the *bootstrap*, and a learner whose target moves with its own parameters has no fixed point to converge to. Inefficiency degrades; a missing fixed point diverges.

Full DQN is itself unstable, peaking at 469 and ending at 250 — 56% of its own best, with a seed standard deviation (151) larger than that gap. The stabilisers make CartPole learnable, not stable. This is worth carrying into A4: the policy-gradient methods there are often described as high-variance relative to value methods, and the honest baseline for that comparison is this number, not the peak.

Item 2 required measuring in the right window. At 30k steps every arm was within noise of zero bias, which reads as "no effect" and is not what is happening. The bias trajectory starts near -80 — the network *under*estimates while it is still learning — and crosses zero only at 16–20k steps. Overestimation is not present from initialisation; it is what $\max$ does once the estimates are accurate enough for its bias to be the dominant error. Measuring before the crossing measures the wrong thing.

At 90k steps Double DQN halves the typical overestimation on stock CartPole (median $+96.7 \to +44.6$) and nearly doubles median return ($90 \to 166$), the predicted direction; on the reward-noised variant the medians are indistinguishable ($+42.0$ vs $+43.0$).

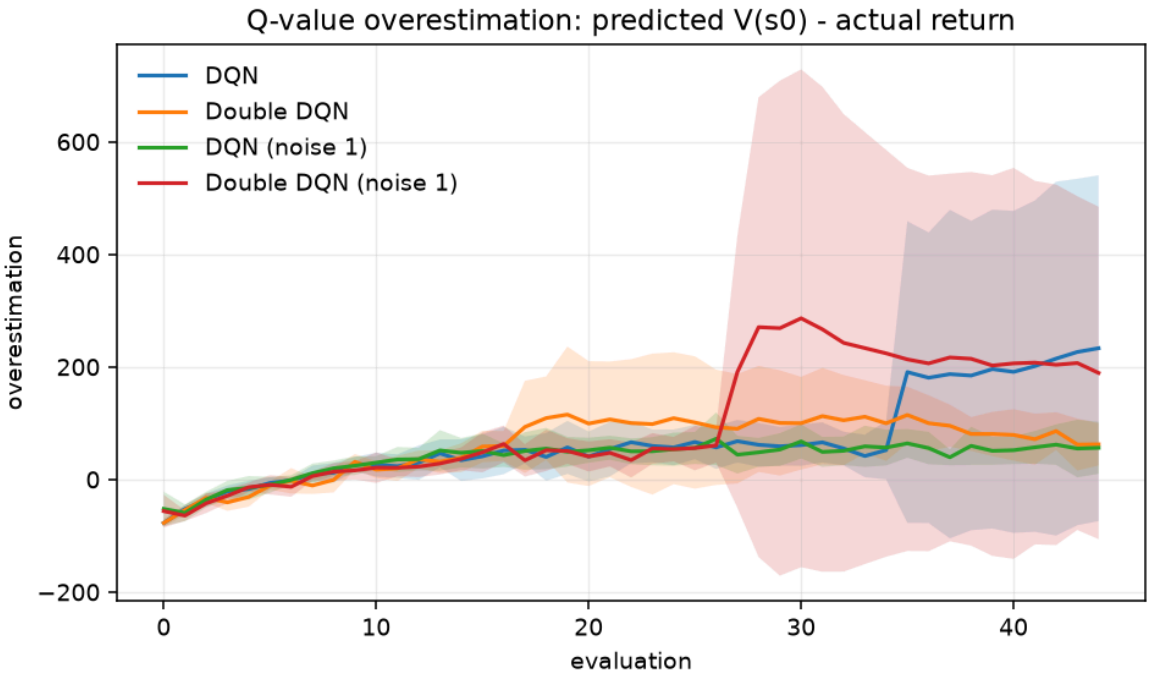


Figure — Predicted $V(s_0)$ minus the realised return. The bias starts negative while the network is learning and turns positive only later; the noised variants show the per-seed divergence discussed below.

The larger effect, though, is per-seed divergence, and it afflicts both. One DQN seed ends at $+841$ and one Double DQN seed at $+779$, against medians in the 20–150 range — and against a ceiling of ≈ 99, the largest discounted return the environment can pay at $\gamma = 0.99$. Estimates that exceed the maximum possible return are not mis-calibration, they are divergence. Double DQN reduces the typical bias; it does not prevent that.

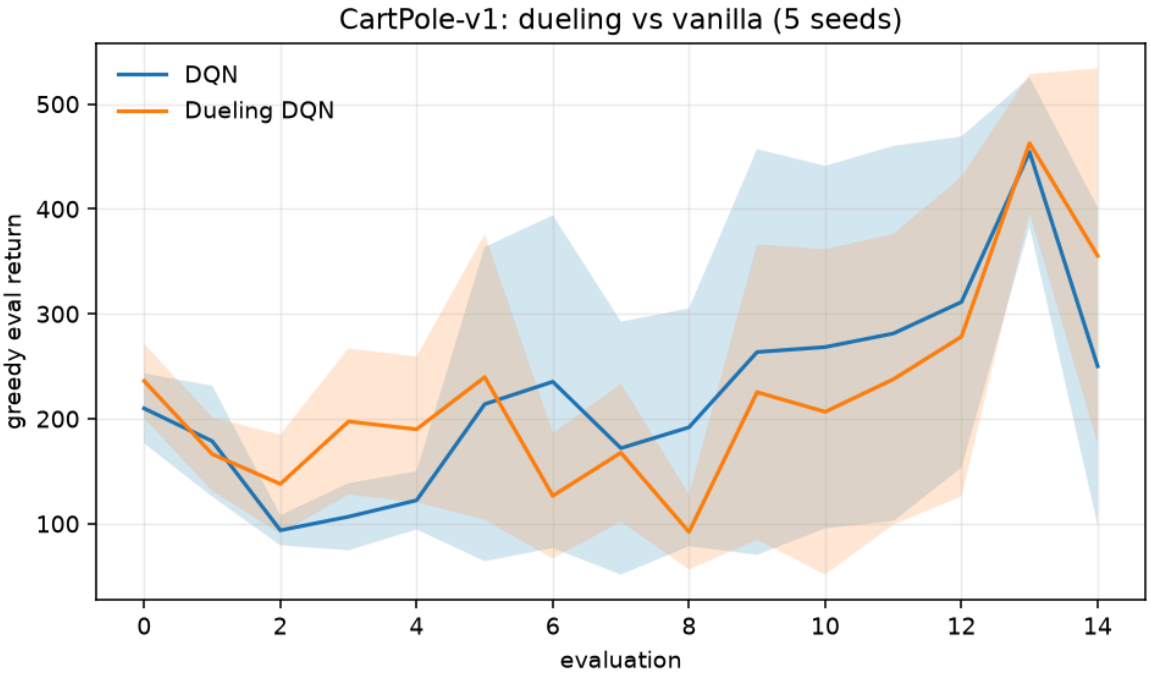


Figure — Dueling against vanilla DQN on CartPole, 5 seeds. The bands overlap throughout.

Item 4 (Atari) settled item 2's question and could not settle item 3's. Run on a Colab T4, Pong, 1M steps per arm, one seed each. The bias probe separates by *sign* across the whole late phase: DQN's estimates exceed its realised returns at 10 of 10 checkpoints past 550k (mean $+0.69$), Double DQN's fall below at 8 of 10 (mean -0.23). This is the prediction from section 4 — the bias needs estimate noise and a horizon before it dominates, and Pong supplies both where CartPole did not. It also matches van Hasselt's result precisely: the double estimator does not eliminate the bias, it **converts overestimation into underestimation**.

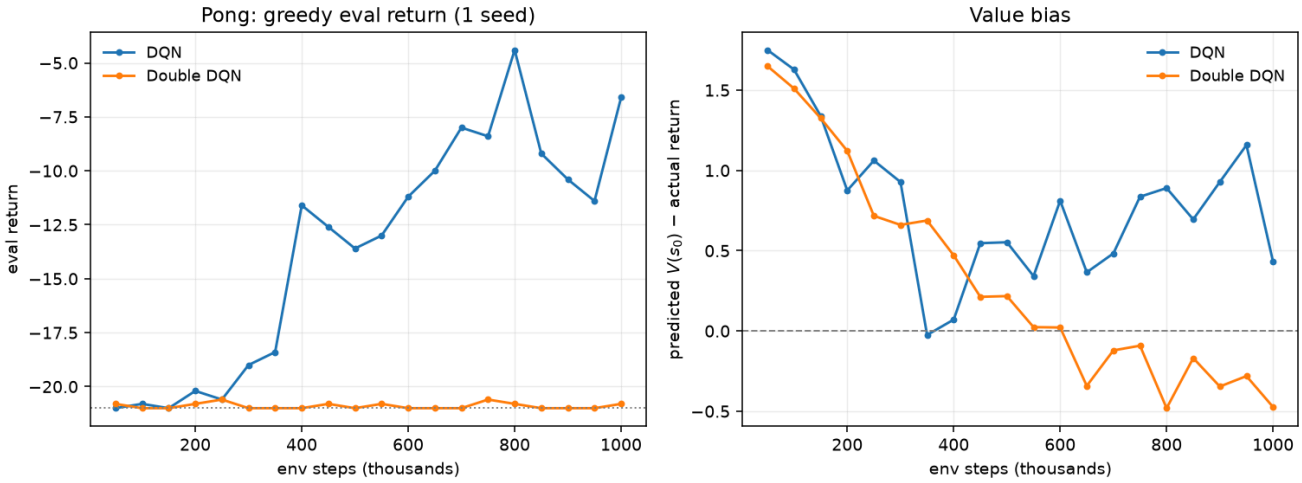


Figure — Pong, one seed per arm: greedy evaluation return (left) and value bias (right). DQN's bias stays positive late in training; Double DQN's goes negative.

The performance comparison, though, is unusable, and that is the more instructive half. Double DQN never learned in this run — 550k consecutive steps at -21 — while an earlier run of the *same code at the same default seed*, lost to a runtime reclaim at 80%, had Double at -14.0 against DQN's -16.0, i.e. ahead. Two nominally identical configurations, opposite orderings, from GPU nondeterminism compounding through a chaotic optimisation. One seed per arm cannot separate a real failure mode from an unlucky trajectory, and this repo's own protocol sets the floor at three; compute (90 GPU-minutes per arm on a runtime reclaimed at ~85) forced the violation, so the result is reported as an illustration rather than a measurement.

And the probe has a flaw that only a large performance gap reveals. It measures $\max_a Q(s_0, a) - \mathbb{E}[\sum_t \gamma^t r_t]$, which is a joint function of value calibration *and* policy quality. With the two arms ending at -6.6 and -20.8, the sign separation cannot be fully disentangled from the fact that one agent learned and the other did not. The correct design compares arms at *matched return* rather than matched step — impossible here, since Double never reached DQN's level. On CartPole the arms tracked closely enough that this never mattered, which is why it went unnoticed through two phases.

Two methodological notes, both learned by getting them wrong first:

- **Means hid the truth twice in this phase.** Reported as means, item 2 says "Double DQN is three times worse under noise" — an artifact of one seed in five. Item 3's median silently excluded seeds that never reached the threshold, flattering the arm

that failed more often. Heavy-tailed outcomes need per-seed values and a median, and both scripts now record them.
- **More training is not more convergence.** DQN's return fell from 250 at 30k steps to 90 at 90k while its value estimates climbed past the achievable maximum. Extrapolating a learning curve forward assumes a stability this algorithm does not have.

8. Where these items come from

item	primary source	why it was picked
1 — DQN, ablating replay and the target network	Mnih et al. 2013 (the NIPS workshop paper: replay, **no** target network) and Mnih et al. 2015 (Nature: adds the target network). The triad itself is S&B §11.3 "The Deadly Triad"	The two papers *are* the ablation. The "no target net" arm is not a strawman someone invented — it is literally the 2013 configuration, and the gap between the arms is the gap between the two publications.
2 — Double DQN	van Hasselt, Guez & Silver 2016; the tabular original is van Hasselt 2010, and S&B §6.7 "Maximization Bias and Double Learning" gives the bandit-sized version of the same bias	It isolates one specific defect — the max operator's upward bias — with a fix small enough that nothing else changes. The measurement, predicted value against realised return, is the paper's own diagnostic.
3 — Dueling architecture	Wang et al. 2016	It is the one improvement in this group that changes the *network*, not the update rule, so it tests whether architecture can buy sample efficiency at a fixed algorithm.
4 — Atari (optional, not run)	Mnih et al. 2015	Initially judged a scale demonstration with no new concept. Items 2 and 3 disproved that: it supplies the value noise Double DQN needs and the large action space dueling needs, so it is where both become measurable. Still needs a GPU.

The **lab** pairing (`Practical_RL` week04_approx_rl) is the ladder's own and covers experience replay, target networks and double/dueling/bootstrap DQN — matching items 1–3.

The **goal statement** in section 2 is quoted from `ladder.md`. The mapping in section 3 from each stabiliser to the triad ingredient it addresses is this repo's framing of the papers' arguments, and item 1 is what tests it — not a claim either paper states in those terms.

9. What this sets up

- **A4** abandons the value-only approach: policy gradients parameterise π_θ directly, which removes the max-operator bias measured in item 2 entirely, at the cost of the variance A1 measured.
- **A5** needs continuous actions, where $\max_a Q(s,a)$ is no longer a lookup over a handful of columns but an optimisation problem — the reason DDPG has an actor at all.

- **B1** meets the replay question again as the on-policy/off-policy tradeoff in PPO: how much can you reuse a batch collected under an older policy before the updates stop being valid.